

CPU workspace / concise review

collision_shpere_generation · robot_workspace_dexterous · robot_workspace_projection

Supported

Full CPU bounded IK and configured collision filtering. Dexterous output includes RWS/DWS, manipulability, condition numbers and singular values. Projection supports task IK, collision-filtered FK, cache rendering and live recomputation. No Torch, cuRobo or Warp import is required on the CPU path.

Optimizations

Skip fixed-body occupied targets before IK; run SVD only for successful solutions; compress snapshots once; cache FCL BVHs and pair exclusions; use accelerated safe YAML parsing. Shared dexterity intersects the same orientation IDs. Projection cache IDs hash model contents and backend settings; points and signatures commit atomically.

Run

Install the sibling collision package first, then both workspace packages with their [cpu] extras. Use `python -B run.py --backend cpu` with the project configuration. Dexterous example: `configs/cpu/spark2_v2.yaml`. Projection example: `config.cpu.yaml`, `--generate-only --no-progress-window`.

Verified

174 regression tests passed. Real CPU presets: Spark2 STL accepted 14/25 cells; dual-arm sphere projection accepted 5/25 left-arm and 3/25 right-arm cells. Separate probes verified cache reuse and an image overlay while prohibiting Torch, cuRobo and Warp imports. Live camera and Qt interaction were not exercised.

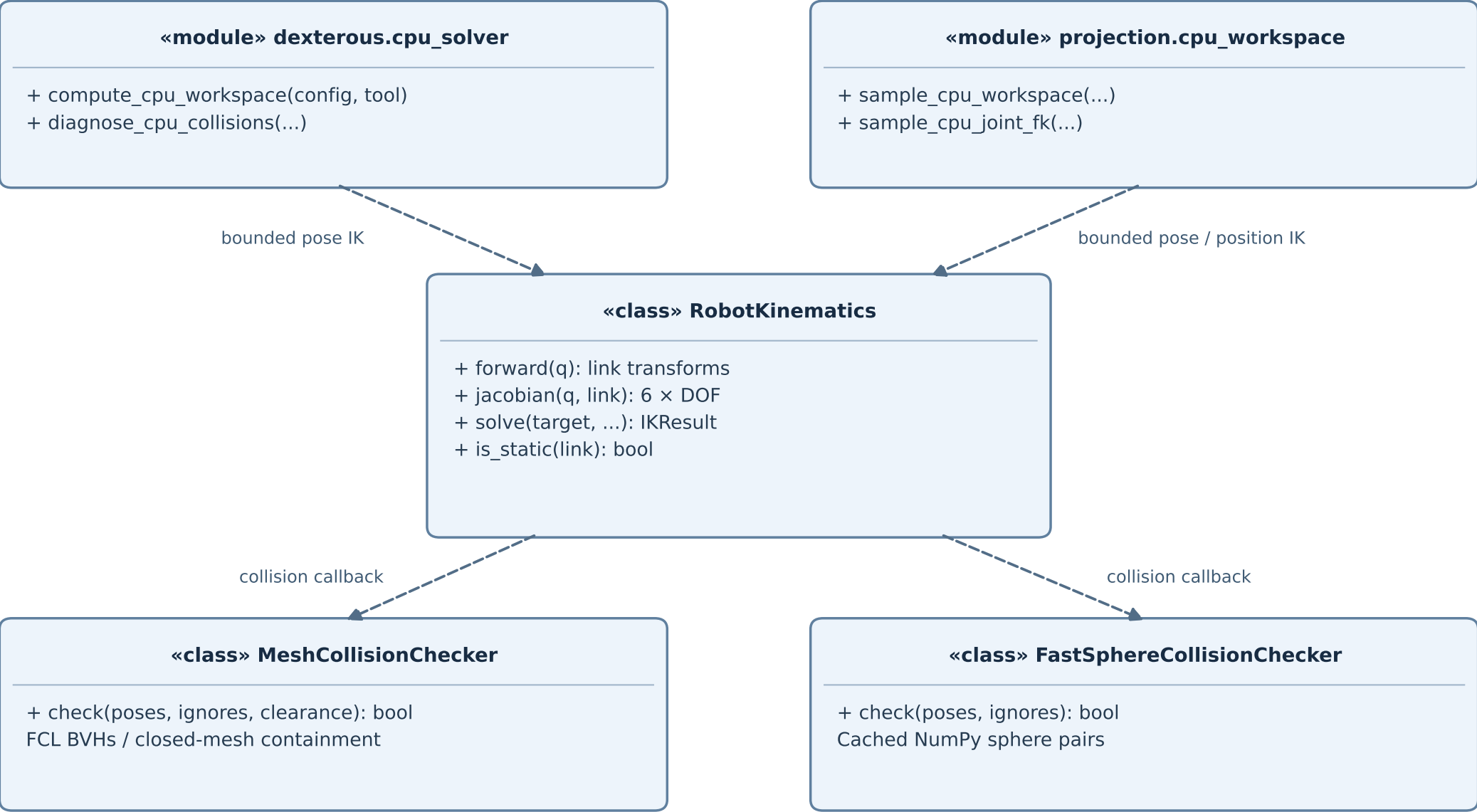
Limits / next work

CPU and CUDA use different numerical searches and can return different sampled acceptance sets. Open meshes provide surface checks; sphere results depend on the supplied approximation. Shared arm results do not solve a simultaneous two-arm constraint. Large grids benefit from CUDA. Next: resumable snapshots, CPU process workers and asynchronous Qt mesh queries.

CPU architecture / UML

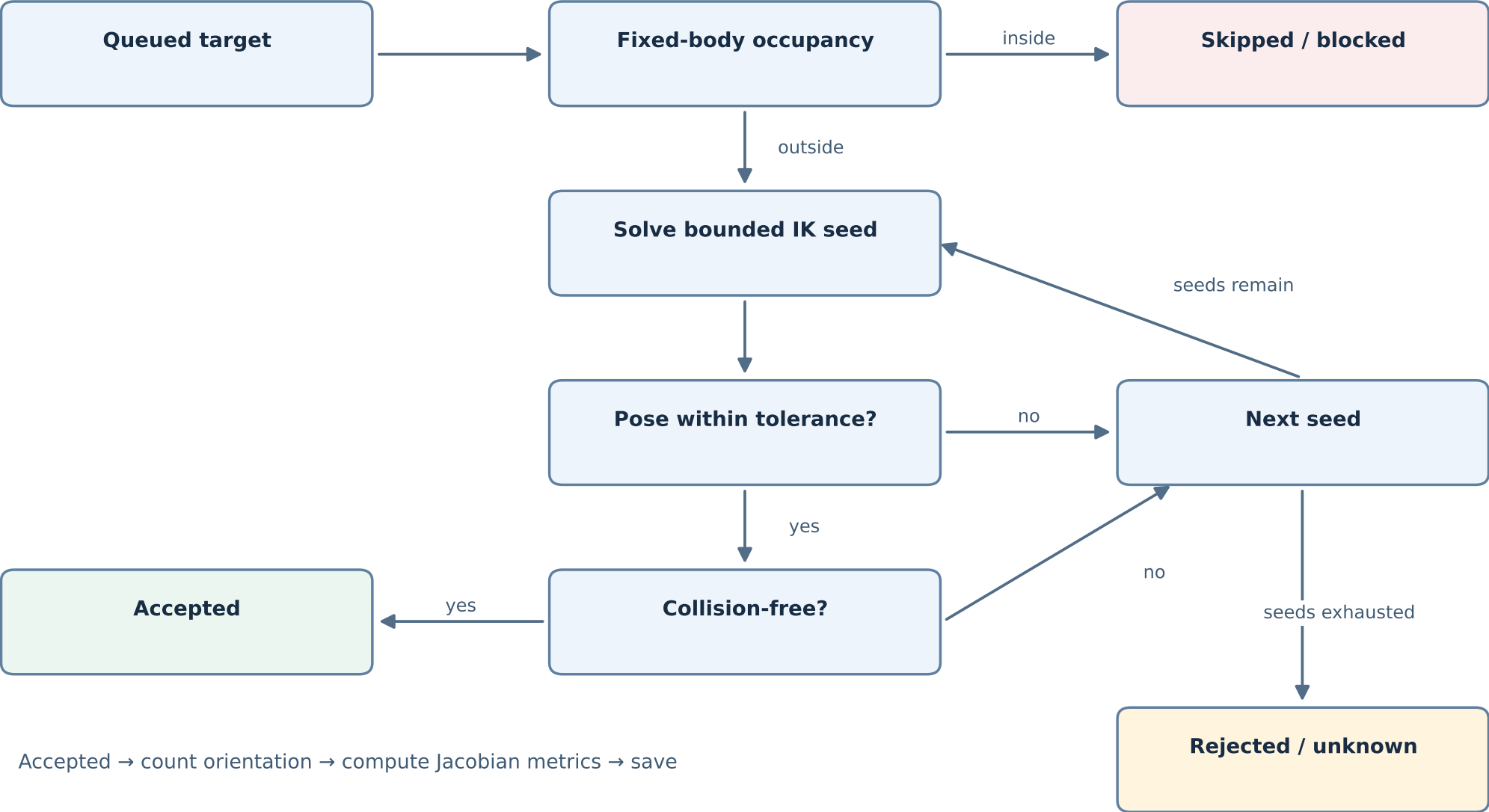
Shared computation, separate application responsibilities. Dashed arrows denote dependencies.

collision_shpere_generation provides the shared classes



Target evaluation / UML state machine

A target is accepted only after pose tolerance and collision checks both pass.



Accepted → count orientation → compute Jacobian metrics → save